Xelulu — Software Team Spec

Xelulu is a screen-free AI companion for kids (ages 1–12). This document lets the **software team build the cloud, the AI/voice pipeline and the parent app in parallel with the hardware**, by fixing the one thing both sides share: the **device gateway API**. Build against the contract (§3) and the simulator (§9); integrate with the real robot later.

---

1. Scope & how we build in parallel

The one rule that lets both teams start today: freeze the device gateway API (§3) first, then build to it. The hardware team builds the robot and the firmware that speaks that API; the software team builds the cloud, the AI/voice pipeline and the parent app against the same API plus a **device simulator** (§9). Neither team waits for the other — they meet at the gateway.

Software team owns (this doc):

- **Cloud backend / gateway** — the service the robot connects to, and the brain behind it (speech-to-text, the LLM conversation with child-safety guardrails, text-to-speech, content delivery, routines, reporting, OTA).
- **Parent mobile app** (iOS + Android) and its backend.
- **Content pipeline** — stories, songs, Grandma recordings, voice management.
- **Device simulator** used to develop and test without a physical robot.

Hardware team owns (separate hardware doc): the robot, its sensors/actuators, and the **firmware** that (a) handles on-device wake-word + cry detection, (b) runs the face/motion, and (c) implements the device side of the gateway API. The firmware is the bridge; the software team treats the robot as a client that speaks the §3 contract.

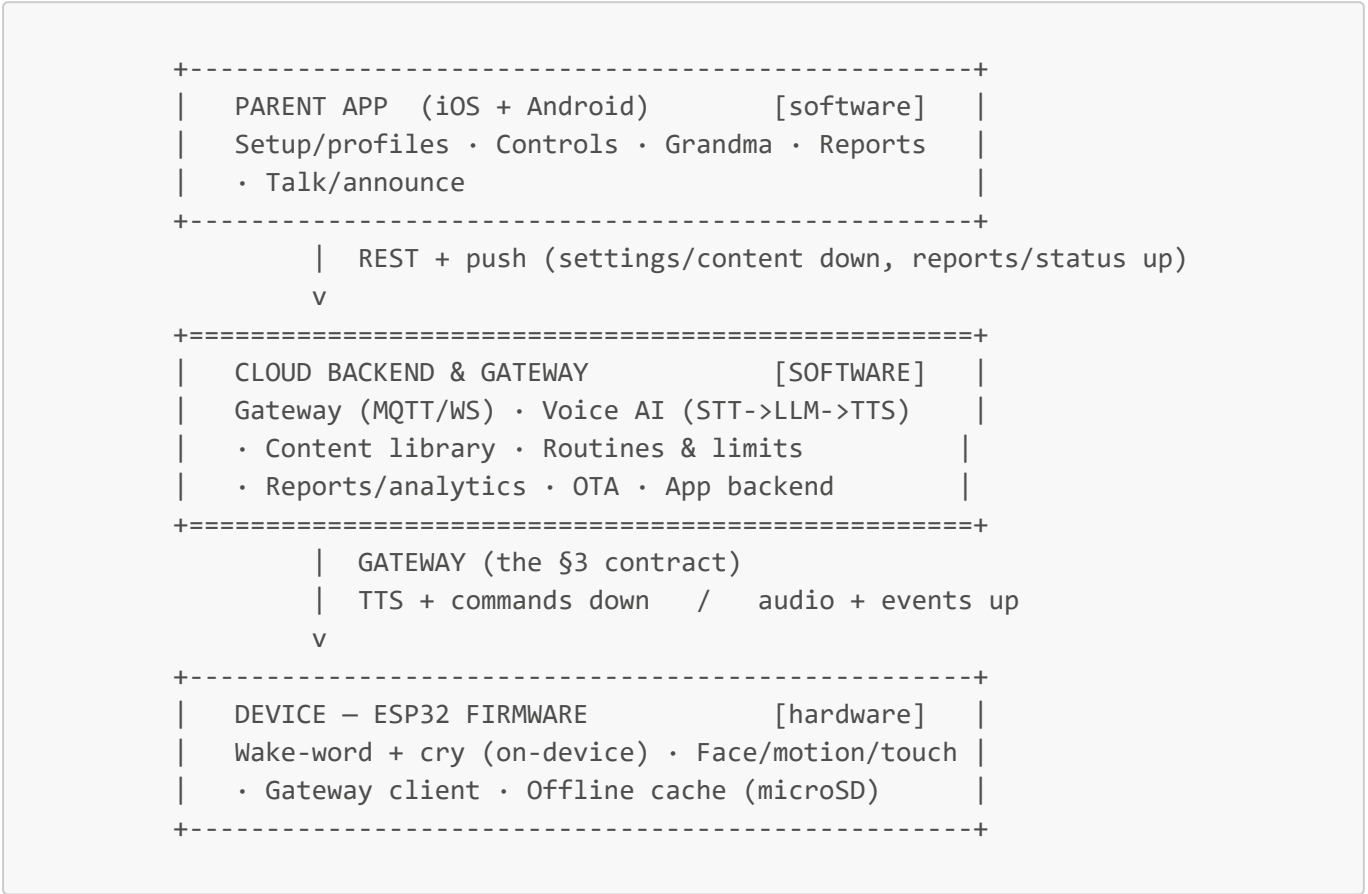
What runs where — the split that matters:

Concern	On device (firmware)	In cloud (software)	In app (software)
Wake word, cry detection	Yes (on-device model)	—	—
Speech-to-text, LLM, TTS	—	Yes	—
Face animation, motion, touch	Yes	Commands only	—
Conversation brain + guardrails	—	Yes	—
Content library (songs/stories)	Cache for offline	Master + delivery	Browse/assign
Routines, timers, limits	Executes	Defines/schedules	Parent sets
Reports, mood moments	Emits events	Aggregates	Parent views
Parental controls	Enforces	Stores/propagates	Parent edits

Everything the software team builds talks to the robot only through the gateway, so the robot can be mocked. The integration milestone (§11) is simply: point the cloud at a real device instead of the simulator.

2. System architecture

Three tiers, one shared boundary. The robot connects **up** to the cloud gateway; the parent app connects to the same cloud. The software team builds the top two tiers; the gateway (§3) is the only interface the device and cloud must agree on.



Two interfaces, two rules:

1. **App ↔ Cloud** — ordinary REST + push notifications. The software team owns both ends, so it can evolve freely.
2. **Cloud ↔ Device (the gateway)** — the frozen contract in §3. It changes only by version bump, because the hardware team builds to it too.

Practical consequence: the software team develops the whole stack against the **device simulator** (§9), which speaks the gateway exactly like the real robot. Integration (§11) is just swapping the simulator for a physical device — no code change above the gateway.

---

3. Device ↔ Cloud gateway API — THE integration contract

This is the section both teams freeze first. It defines the boundary so each side can build and test against a stub of the other.

3.1 Transport & auth

Property	Decision	Notes
Protocol (control)	**MQTT 5 over TLS**	Lightweight, pub/sub, survives flaky Wi-Fi, auto-reconnect
Protocol (audio)	**WebSocket (binary) over TLS**	Opened only during a conversation turn; low-latency duplex audio
Auth	Per-device **X.509 client cert** (or a device JWT) provisioned at factory	Each robot has a unique `device_id`
Identity	`device_id` (immutable), bound to one `household_id` at pairing	
Versioning	`proto_ver` field in every message	Mismatch → device stays on last-known-good, reports it
Heartbeat	Device publishes status every 15 s	Cloud marks offline after 45 s
QoS	Commands QoS 1 (at-least-once, idempotent via `cmd_id`)	Telemetry QoS 0/1

3.2 MQTT topics

```
xelulu/{device_id}/status      device -> cloud   heartbeat, battery, mode, fault
xelulu/{device_id}/event       device -> cloud   touch, wake, cry, motion, button,
errors
xelulu/{device_id}/cmd         cloud  -> device  say/animate/move/play/led/config
xelulu/{device_id}/ack         device -> cloud   command results (by cmd_id)
xelulu/{device_id}/ota         cloud  -> device  update availability + manifest
```

Audio uses a separate signalled WebSocket session (see §7), not MQTT.

3.3 Message schemas (JSON)

status (device → cloud), every 15 s

```
{ "proto_ver": 1, "ts": 1733400000, "battery_pct": 72, "charging": false,
  "mode": "idle", "wifi_rssi": -58, "fw_ver": "1.4.2",
  "fault": [], "active_child_id": "c_01" }
```

event (device → cloud) — one envelope, `type` discriminates:

```
{ "proto_ver": 1, "ts": 1733400001, "type": "touch",
  "data": { "zone": "belly", "gesture": "double_tap" } }
```

`type` values and their `data`:

- `touch` → `{ zone: head|back|belly|left_hand|right_hand, gesture: tap|double_tap|hold|pat }`
- `wake` → `{ source: "hey_lulu" | "touch", dir_deg: 35 }` (sound direction, if known)
- `cry` → `{ confidence: 0.0-1.0, duration_s: 7 }` (NO audio attached)
- `motion` → `{ event: pickup|shake|upside_down|drop|tilt, value: {...} }`
- `button` → `{ button: "belly_talk", action: press|release }`
- `nfc` → `{ card_id: "story_042" }` (if NFC fitted)
- `error` → `{ code, detail }`

cmd (cloud → device) — one envelope, `action` discriminates, every cmd has a `cmd_id`:

```
{ "proto_ver": 1, "cmd_id": "9f2c", "action": "say",
  "data": { "audio_url": "wss://.../turn/abcd", "expression": "happy",
            "duration_hint_ms": 2200 } }
```

`action` values and their `data`:

- `say` → stream/play TTS audio + sync a face expression (audio via the WS turn)
- `animate` → `{ expression: happy|love|sleepy|curious|sad|listening|..., eyes: "blink"|"hearts"|..., loop: false }`
- `gesture` → `{ name: wave|high_five|hug|clap|point_left|nod|shake|... }` (arms + head)
- `move` → `{ cmd: come|stop|turn, linear_mms: 150, angular_mrad_s: 0, max_dist_mm: 2000 }`
- `play` → `{ content_id: "song_12", source: "stream"|"cache" }`
- `set_mode` → `{ mode: idle|play|story|quiet|sleep }`
- `config` → push a settings delta (volume, quiet hours, movement on/off, language, `active_child_id`)
- `ota` → `{ manifest_url }`

ack (device → cloud)

```
{ "proto_ver": 1, "cmd_id": "9f2c", "status": "done", "detail": "" }
```

`status`: `accepted` | `done` | `rejected` | `failed`. Hardware is the source of truth for whether a command physically completed.

3.4 What each team stubs

- **Software builds against the simulator (§9)** — it publishes realistic `status`/`event`, accepts `cmd`, returns `ack`, and plays a fake audio turn.
- **Hardware builds against a mock cloud** — a tiny MQTT broker + script that emits `cmd` and logs `event`/`ack`.

Freeze §3.1–§3.3 by sign-off from both leads before coding. Changes go through a version bump of `proto_ver`.

---

4. Cloud backend services (software team build)

Service	Responsibility	Key integrations
Gateway	MQTT/WS broker + device registry, auth, routing, session state	Device (§3), all services
Voice AI pipeline	Wake→STT→LLM(+guardrails)→TTS orchestration per turn	STT, LLM, TTS vendors
Conversation/brain	Child-level persona, memory, why-chains, ask-a-grown-up deferral, multilingual	LLM, content, safety
Content library	Stories, songs, Grandma recordings, packs; delivery + caching manifests	CDN, device cache
Safety/guardrails	Topic classification, age-appropriate filter, deferral routing, profanity/strangers	Conversation
Routines & limits	Schedules (bedtime, brushing), play-time limits, quiet hours	Device config, app
Reports/analytics	Aggregate events into daily/weekly reports, mood moments, words learned	Events, app
OTA	Firmware + content update manifests, staged rollout, rollback	Device
App backend	Accounts, households, profiles, auth, push, Grandma invites	App

P0 vendor choices (swappable behind interfaces): STT and TTS as streaming cloud APIs with Indian-language support; an LLM with function-calling for the brain. Keep each behind an internal interface so they can be swapped. Multilingual = English + 10 Indian languages (per catalogue).

---

5. Parent mobile app (iOS + Android)

From the catalogue §4.12, plus talk/Grandma. Screens:

Area	Functions
Setup	Pair device, Wi-Fi, child profiles (name, pronunciation recording, age stage, languages), Grandma language/name, camera enable (N/A for Xelulu P0), household members & roles
Home	Battery, online status, today's play time, last activity, charge reminder
Parental controls	Daily play time per child, quiet hours, bedtime, volume limit, movement on/off, allowed content categories, topics to always defer, second-language choice
Grandma	Invite grandparents (Grandma role); record stories/messages; schedule bedtime stories; live Grandma call; voice consent + Grandma-voice management
Reports	Daily summary, weekly learning report, words learned, stories heard, questions asked (summaries), deferred questions, crying episodes, monthly memory book

Area	Functions
Talk	Walkie-talkie, announcements
Play	Remote drive (joystick), start a game/dance/story remotely, story-card manager
Content	Story library, songs, packs, card packs, costume skins, festival specials
Privacy	Memory review & delete, transcript access (30-day default), delete all data, export, consent records

Roles: **Parent** (full), **Caregiver/nanny** (start games/routines, see battery; no reports/controls), **Grandma** (record + call only).

---

6. Data models (shared schemas)

```
Household   { household_id, owner_user_id, members[], devices[], created_at }
User        { user_id, role: parent|caregiver|grandma, auth, push_token }
ChildProfile{ child_id, household_id, name, name_pronunciation_audio,
              age_stage: tiny|explorer|maker, languages[], second_language,
              content_categories[], defer_topics[], limits{...} }
Device      { device_id, household_id, fw_ver, last_seen, active_child_id }
Session     { session_id, device_id, child_id, started_at, ended_at, events[] }
Event       { event_id, device_id, child_id, ts, type, data }   // mirrors §3
events
ContentItem { content_id, type: story|song|rhyme|card, languages[],
              audio_urls{}, duration_s, age_stages[], cacheable }
GrandmaAsset{ asset_id, household_id, grandparent_user_id, kind, audio_url,
              consent_record_id, schedule? }
Consent     { consent_id, household_id, type, granted_by, ts, scope }
Report      { report_id, child_id, period, summary, words_learned[],
              stories_heard[], questions[], mood_moments[] }
```

`child_id` + `active_child_id` let one device serve multiple children; the cloud tells the device who is active via a `config` command.

---

7. Voice & conversation pipeline

Wake word and cry detection run **on-device** (no always-on audio to cloud). A conversation turn:

1. **Wake** — device detects "Hey Lulu" / touch-to-talk → publishes a `wake` event and opens the **audio WebSocket** turn.
2. **Stream up** — device streams mic audio (Opus/PCM frames) to the cloud over the WS.
3. **STT** — cloud streams partial transcripts; endpointing closes the utterance.
4. **Brain + guardrails** — transcript → LLM with the child persona, profile, memory, and the safety filter. Sensitive topics → **ask-a-grown-up deferral** reply + a flag to the app.
5. **TTS** — the reply is synthesized (chosen language/voice) and streamed **back down** the WS.

6. **Play + express** — a `say` command syncs the face expression; the device plays the audio and animates; closes the turn.

Latency budget (target, wake→first audio out): ≤ 1.2 s. Stream STT and TTS (don't wait for full turns); keep the WS warm during a session; pre-warm the LLM. If the network stalls mid-turn, the device falls back to a cached "let me think..." filler and, on timeout, a graceful offline line.

Privacy: raw audio is processed in-flight and **not stored**; only text summaries + event metadata persist (see §10).

---

8. Offline mode contract

Must work with **no internet** (firmware + cached content; cloud does nothing here): wake word & touch, all touch/motion reactions, copycat voice, 20 dances, freeze dance, Xelulu Says, 30 cached songs + 30 cached stories in the child's languages, story cards (cached audio), code cards, cry detection & comfort (**parent alert queued, sent when internet returns**), routines & timers, charge reminders by voice.

Offline voice line: *"My star library is sleeping right now, but we can still play and dance!"*

Re-sync on reconnect: the device flushes its queued `event`s (including offline cry episodes with timestamps) to the cloud; the cloud returns any pending `config`/content/OTA. Events carry device timestamps (RTC, synced when online) so reports stay ordered. The cloud must accept **back-dated events** idempotently (dedupe by `event_id`).

---

9. Device simulator / mock (build without hardware)

The software team's most important enabler. A small service (Node/Python) that **speaks the §3 gateway exactly like the robot**:

- Connects to the gateway with a fake `device_id` + test cert.
- Publishes realistic `status` (battery drain, charging) and `event`s (touch gestures, wake, cry, motion) — scriptable and via a tiny web UI (buttons per touch zone, "say this", "trigger cry").
- Accepts every `cmd`, logs it, and returns `ack` (`accepted`→`done`), simulating durations.
- Runs a **fake audio turn**: accepts the WS, plays a canned transcript, echoes TTS as a tone/length so latency can be measured.
- Has an **offline toggle** to test queue-and-resync (§8).

Deliverable alongside the backend; it is the integration test harness (§11). Keep it in the repo so CI can run end-to-end tests without hardware.

---

10. Safety, privacy & compliance (kids)

Non-negotiable for a children's product:

- **No raw audio stored.** Audio is processed in-flight; only text summaries + event metadata persist. Cry detection sends an **alert, never audio**.
- **Consent & data rights.** Explicit parental consent at setup; Grandma-voice needs a recorded consent statement; app provides transcript access (30-day default), delete-all, and export. Treat as

COPPA/GDPR-K style — data minimization, parent-controlled.

- **Content guardrails.** Age-appropriate filter on every LLM reply; sensitive topics (death, body, violence, religion, family conflict) → gentle age-appropriate answer + "let's ask Amma/Appa together," and surfaced in the app.
- **No open internet to the child.** The child only reaches the curated brain + content; no web browsing, no third-party chat.
- **Household-locked voices & content.** Grandma voice usable only within that household.
- **Safe-flashing / wellbeing** handled on device (no 3–30 Hz flashing), enforced in firmware; cloud respects quiet hours and play-time limits.
- **PII handling.** Child name/pronunciation and recordings are encrypted at rest; access scoped by household + role.

---

11. Milestones & parallel delivery plan

Software workstreams that proceed with **zero hardware**, then integration gates.

Phase	Software (no hardware needed)	Gate
M0 — Contract freeze	Both leads sign off §3 (gateway + schemas)	§3 frozen, `proto_ver` 1
M1 — Simulator + gateway	Device simulator (§9) + gateway/broker + device registry/auth	Sim connects, cmd/event/ack round-trip
M2 — Voice pipeline	Wake→STT→LLM(+guardrails)→TTS over the WS turn; latency ≤1.2 s against sim	Live conversation via simulator
M3 — Content + routines	Content library + delivery/caching manifests, routines/limits, reports	Offline cache manifest + report from sim events
M4 — Parent app	App (setup, controls, Grandma, reports, talk) on app backend	Parent can drive the simulator end-to-end
M5 — Safety & privacy	Guardrails, consent flows, data rights, deferral routing	Safety test suite green
M6 — Integration	Swap simulator for a real robot (hardware reaches its own M-gate)	Real device does a full conversation + play session
M7 — Pilot	OTA pipeline, telemetry dashboards, field fixes	Stable multi-day runtime on real units

Integration (M6) is low-risk **because** everything above the gateway was built and tested against the simulator — the only new variable is the physical device honoring the same contract.

This spec covers Xelulu (ages 1–12). Sathi/WithU, Parth and Sarthi reuse the same platform, gateway and cloud; their differences are feature config + device capabilities, not new interfaces.